

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS
SUPERIORES DE MONTERREY

ESCUELA DE INGENIERÍA Y CIENCIAS

Evidencia Final: Reporte Reto

Dispositivo de medición portátil de luminancia para pantallas de
Carrier



**Tecnológico
de Monterrey**

Curso: Caracterización experimental mediante instrumentación
electrónica

Grupo: 301

Integrantes:

José Feliciano Gutiérrez Rubio | A01707914

José Alfredo Almaguer Cruz | A00839015

Héctor Enrique Campbell Salas | A00839531

Bernardo Lugo Albor | A01707191

Junio 2026

Índice

1. Introducción	2
1.1. Descripción del Reto	2
1.2. Responsabilidades y Tareas Asignadas	2
1.3. Marco Teórico	3
1.3.1. Iluminancia	3
1.3.2. Luminancia	4
1.3.3. Ángulo sólido de una apertura circular	4
1.3.4. Conversión de iluminancia a luminancia	4
2. Desarrollo de la Solución	5
3. Prototipado	6
4. PCB del Hardware Propuesto	8
5. Descripción del Hardware	10
6. Explicación del Software desarrollado	13
6.1. Aplicación de escritorio	15
6.1.1. Librerías utilizadas	15
6.1.2. Cinemática del brazo	16
6.1.3. Capa de transporte (USB / WiFi)	18
6.1.4. Estructura de la interfaz	18
6.1.5. Secuencias automáticas y medición	20
6.2. Firmware del Arduino	20
6.2.1. Librerías utilizadas	20
6.2.2. Funcionamiento del programa	20
6.3. Conceptos de Arduino y comunicaciones	21
7. Caracterización Experimental del Sensor de Iluminancia	22
7.1. Respuesta no lineal de la pantalla	22
7.2. Linealización logarítmica	24
7.3. Validación del sensor con una fuente patrón de LED	25
7.4. Parámetros metrológicos del sensor	27
8. Conclusiones	27
8.1. Conclusiones individuales	28
A. Códigos desarrollados	32
B. Desarrollo matemático de la cinemática del brazo de dos grados de libertad	32

1. Introducción

La empresa Carrier ha designado la tarea de medir luminancia en pantallas digitales del tipo: Transicold APX [1], con el propósito de observar su comportamiento en entornos de iluminación variables.

Por ello, resulta de especial interés contar con una herramienta capaz de verificar de forma objetiva si el nivel de luz emitido por una pantalla se encuentra dentro de un rango aceptable.

En el presente reporte se presenta el diseño y desarrollo un brazo robótico, capaz de medir luminancia para cualquier tipo de pantallas digitales.

El sistema está basado en una placa Arduino UNO R4 WiFi y emplea un sensor de luz ambiental VEML7700 para medir la luminancia de pantallas digitales, indicando el estado mediante un sistema de LEDs rojo, amarillo y verde, mostrando las lecturas en una pantalla LCD y transmitiendo los datos a la plataforma en la nube ThingSpeak.

Además, para mejorar la experiencia del usuario, se ha integrado un brazo robótico de 2 grados de libertad, el cual es capaz de posicionar el sensor de forma reproducible en cada punto de medición.

1.1. Descripción del Reto

El reto consiste en diseñar y desarrollar un prototipo capaz de medir la luminancia de pantallas digitales y determinar si esta se encuentra dentro de un rango aceptable. La solución debe ser portátil, de bajo costo y operable en distintos entornos de medición.

Es importante distinguir las dos magnitudes fotométricas involucradas en el problema, ya que se relacionan pero no son equivalentes:

- La iluminancia (E) es el flujo luminoso incidente sobre una superficie y se mide en lux (lx).
- La luminancia (L) es la intensidad luminosa emitida o reflejada hacia el observador y se mide en cd/m^2 .

La legibilidad de una pantalla depende de la luminancia percibida por el ojo, no de la iluminancia que recibe la superficie. Ambas magnitudes se relacionan a través del ángulo sólido Ω bajo el cual el sensor observa la fuente:

$$L = \frac{E}{\Omega}. \quad (1)$$

El sensor VEML7700 entrega una lectura de iluminancia en lux; aplicando la relación (1) es posible estimar la luminancia de la pantalla caracterizada.

1.2. Responsabilidades y Tareas Asignadas

A continuación se describen las responsabilidades y tareas asignadas a cada integrante del equipo durante el desarrollo del reto.

Tabla 1: Distribución de responsabilidades del equipo.

Integrante	Tareas y responsabilidades
José Feliciano Gutiérrez Rubio	Desarrollo del software del sistema, incluyendo tanto el código ejecutado en el Arduino como la aplicación de control en la PC.
José Alfredo Almaguer Cruz	Diseño del hardware del sistema, incluyendo el circuito electrónico, integración de componentes y conexiones eléctricas.
Héctor Enrique Campbell Salas	Diseño de la PCB y diseño e impresión de las piezas estructurales en 3D.
Bernardo Lugo Albor	Caracterización experimental del sistema y análisis de resultados.

1.3. Marco Teórico

En esta sección se presentan las magnitudes fotométricas fundamentales y la relación geométrica necesaria para estimar la luminancia de una fuente a partir de las lecturas de iluminancia obtenidas con el sensor VEML7700, considerando el ángulo sólido definido por la apertura óptica del sistema.

1.3.1. Iluminancia

La iluminancia E se define como la densidad superficial del flujo luminoso incidente sobre una superficie, es decir, la razón entre el flujo luminoso Φ que incide sobre un elemento de superficie y el área dA de dicho elemento [2, 3]:

$$E = \frac{d\Phi}{dA} \quad (2)$$

La unidad de la iluminancia en el Sistema Internacional es el lux ($\text{lx} = \text{lm}/\text{m}^2$). En el sistema desarrollado, la iluminancia es la magnitud que el VEML7700 entrega de forma directa, a partir de la cuenta del registro ALS y del factor de resolución asociado a la combinación de ganancia y tiempo de integración configurados, y representa la cantidad de luz que llega al plano del sensor proveniente de todas las direcciones dentro de su campo de visión.

1.3.2. Luminancia

La luminancia L se define como la densidad de intensidad luminosa por unidad de área proyectada, en una dirección específica, de una superficie real o imaginaria [2, 3]:

$$L = \frac{d^2\Phi}{dA d\Omega \cos\theta} = \frac{dI}{dA \cos\theta} \quad (3)$$

donde dI es la intensidad luminosa emitida o reflejada por el elemento de área dA en la dirección de observación, $d\Omega$ es el ángulo sólido elemental que subtiende dicha dirección y θ es el ángulo entre la normal a la superficie y la dirección de observación. Su unidad es la candela por metro cuadrado (cd/m^2). A diferencia de la iluminancia, que describe la luz que incide sobre el sensor, la luminancia describe el brillo de la fuente observada tal como sería percibido visualmente, y es la magnitud que el sistema busca estimar para cada orientación del brazo robótico portasensor.

1.3.3. Ángulo sólido de una apertura circular

El ángulo sólido Ω subtendido por una apertura circular de radio a , observada desde un punto situado sobre su eje a una distancia d , está dado por la expresión exacta [3]:

$$\Omega = 2\pi \left(1 - \frac{d}{\sqrt{d^2 + a^2}} \right) = 2\pi(1 - \cos\theta) \quad (4)$$

donde $\theta = \arctan(a/d)$ es el semiángulo del cono que subtiende la apertura. Cuando $a \ll d$ (aproximación de ángulo pequeño), la expresión anterior se reduce a:

$$\Omega \approx \frac{\pi a^2}{d^2} = \frac{A}{d^2} \quad (5)$$

siendo $A = \pi a^2$ el área de la apertura. Esta aproximación es válida cuando la apertura efectiva del sensor VEMML7700 (o del diafragma colocado frente a él) es pequeña en comparación con la distancia a la fuente de luz observada, condición que define directamente el campo de visión angular del sistema de medición.

1.3.4. Conversión de iluminancia a luminancia

Para una fuente extensa cuya luminancia puede considerarse aproximadamente uniforme dentro del ángulo sólido Ω que subtiende vista desde el detector, la luminancia de la fuente se relaciona con la iluminancia E medida en el plano del sensor a través de [3]:

$$E \approx L \Omega \quad \implies \quad L \approx \frac{E}{\Omega} \quad (6)$$

Esta relación se obtiene a partir de la definición de iluminancia como la integral de la luminancia ponderada por el ángulo sólido y por el coseno del ángulo de incidencia; para ángulos

pequeños $\cos \theta \approx 1$, de modo que la integral se reduce al producto $L \Omega$. En el sistema desarrollado, esta aproximación permite convertir la iluminancia (lux) reportada por el VEML7700 en una estimación de la luminancia (cd/m^2) de la fuente observada, utilizando el ángulo sólido definido por la geometría de la apertura óptica del sistema, sin requerir un fotómetro de luminancia dedicado.

2. Desarrollo de la Solución

La solución propuesta integra hardware, software y una estructura mecánica en un único sistema portátil. El funcionamiento general puede resumirse en las siguientes etapas:

1. Desde una aplicación de escritorio en la PC, el usuario selecciona el punto de la pantalla que desea medir (por coordenadas, por una secuencia automática de barrido o haciendo clic en el simulador).
2. La aplicación resuelve la cinemática inversa del brazo de 2-GDL y calcula los ángulos de los servomotores necesarios para alcanzar ese punto.
3. Los ángulos se transmiten al Arduino mediante comunicación serial (USB) o por red (WiFi/TCP), y el Arduino posiciona el sensor.
4. Una vez posicionado, el Arduino toma n muestras con el sensor VEML7700, calcula el promedio y lo devuelve a la PC.
5. El valor medido se indica localmente con los LEDs y la LCD, se grafica en la aplicación y, opcionalmente, se publica en ThingSpeak y se exporta a un archivo CSV.

La interacción entre la aplicación de escritorio y el prototipo físico durante la ejecución del sistema puede observarse en la Figura 1. En ella se aprecia cómo la aplicación permite controlar el brazo robótico, visualizar las mediciones obtenidas y supervisar el estado general del proceso de caracterización.



Figura 1: Sistema completo en funcionamiento. La aplicación de escritorio se comunica con el Arduino para posicionar el sensor de iluminancia, adquirir mediciones y visualizar los resultados obtenidos.

Entre las principales ventajas de la solución desarrollada destacan que se trata de un sistema automático y simple de operar, que permite controlar todo el proceso desde una aplicación propia, que es adaptable a distintos entornos de medición de luminancia y que logra cubrir el área de trabajo empleando únicamente dos grados de libertad. Como se observa en la Figura 1, la integración del hardware y software en una misma plataforma facilita la adquisición de datos y la supervisión de las mediciones en tiempo real.

3. Prototipado

El prototipo se construyó sobre una estructura impresa en 3D que aloja la placa Arduino, el brazo robótico, el sensor y los elementos de indicación. Las distintas piezas estructurales (base, soportes de los servomotores, eslabones del brazo y carcasa) fueron diseñadas mediante software CAD y manufacturadas por impresión 3D en PLA, lo que permitió iterar rápidamente sobre la geometría y obtener un montaje ligero y portátil. Las vistas CAD del diseño final se muestran en las Figuras 2 y 3.

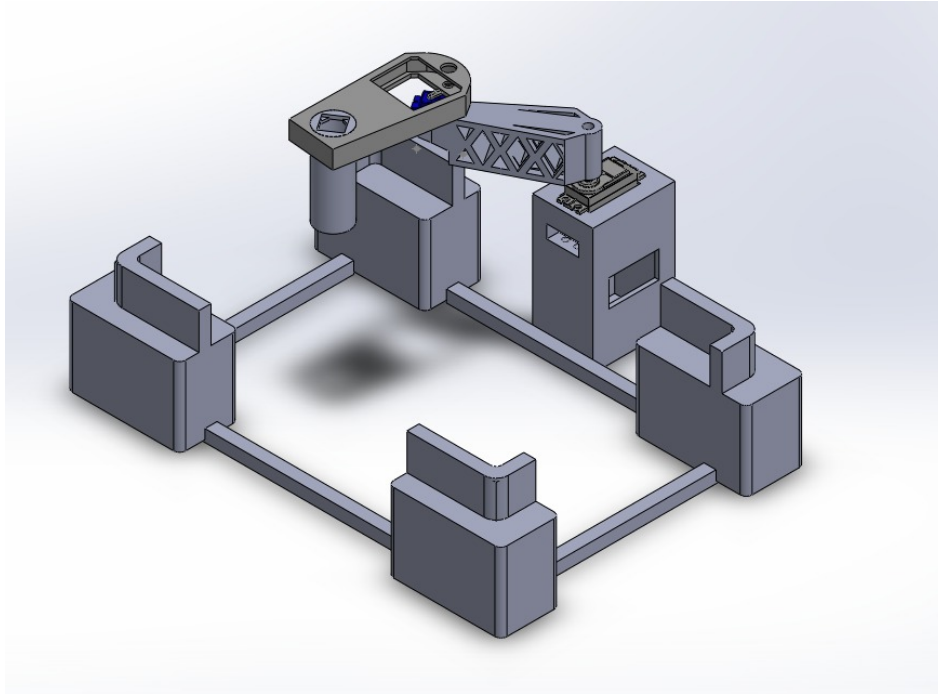


Figura 2: Vista isométrica del modelo CAD del prototipo, mostrando la disposición general de la estructura, los servomotores y el brazo robótico de dos grados de libertad.

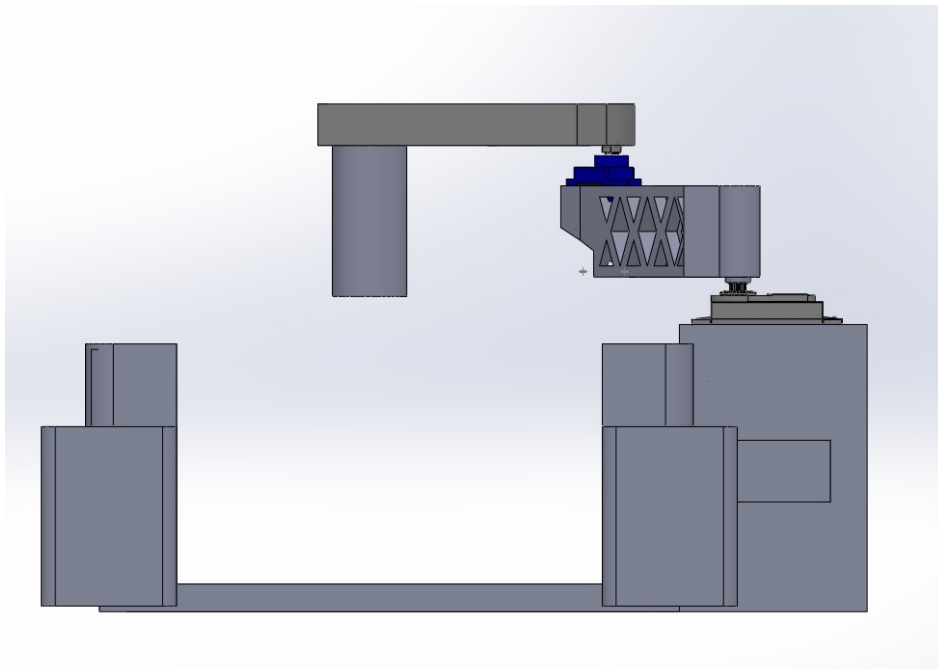


Figura 3: Vista lateral del modelo CAD del prototipo, utilizada para verificar dimensiones, rangos de movimiento y distribución de componentes.

El sistema completo se alimenta mediante baterías, lo que refuerza su carácter portátil:

una batería de 9 V alimenta la placa Arduino y un arreglo de 4 pilas AA (6 V) alimenta los servomotores de forma independiente, evitando que los picos de corriente de los motores afecten la electrónica de control y de medición. La estructura física ensamblada puede observarse en la Figura 4.

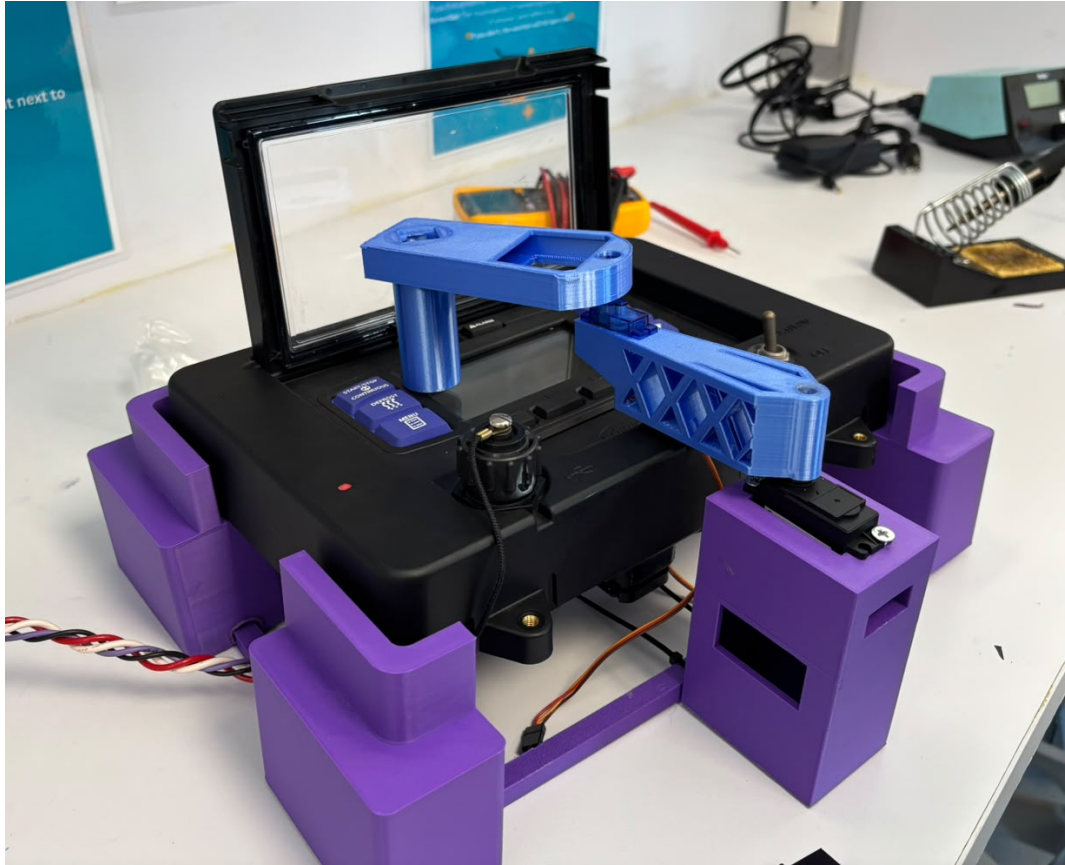


Figura 4: Prototipo físico ensamblado con la estructura impresa en 3D y el brazo robótico montado. En esta etapa aún no se encuentra instalado el sensor de medición en el extremo del brazo.

4. PCB del Hardware Propuesto

Con el fin de dar robustez y permanencia a las conexiones del prototipo, se diseñó una placa de circuito impreso (PCB) que reemplaza el cableado sobre protoboard. La PCB se diseñó como un shield que se monta directamente sobre la placa Arduino UNO R4. Aunque el diseño fue completado durante el desarrollo del proyecto, la manufactura de la tarjeta no pudo realizarse dentro del tiempo disponible para la práctica. Por ello, la PCB propuesta se plantea como una mejora futura orientada a incrementar la confiabilidad mecánica y eléctrica del sistema.

La placa consta de dos capas: una destinada principalmente a la tierra común del sistema y otra para las conexiones de corriente y de señales. El diseño de enrutamiento de ambas

capas se presenta en la Figura 5. En dicha figura, las pistas de la capa superior se muestran en color rojo, mientras que las correspondientes a la capa inferior se muestran en color azul. Este diseño permite distribuir adecuadamente las señales de alimentación, control y comunicación, manteniendo una tierra común para todos los componentes del sistema. Asimismo, se incorporan resistencias pull-up en las líneas SDA y SCL, necesarias para el correcto funcionamiento de la comunicación I2C compartida por el sensor y la LCD.

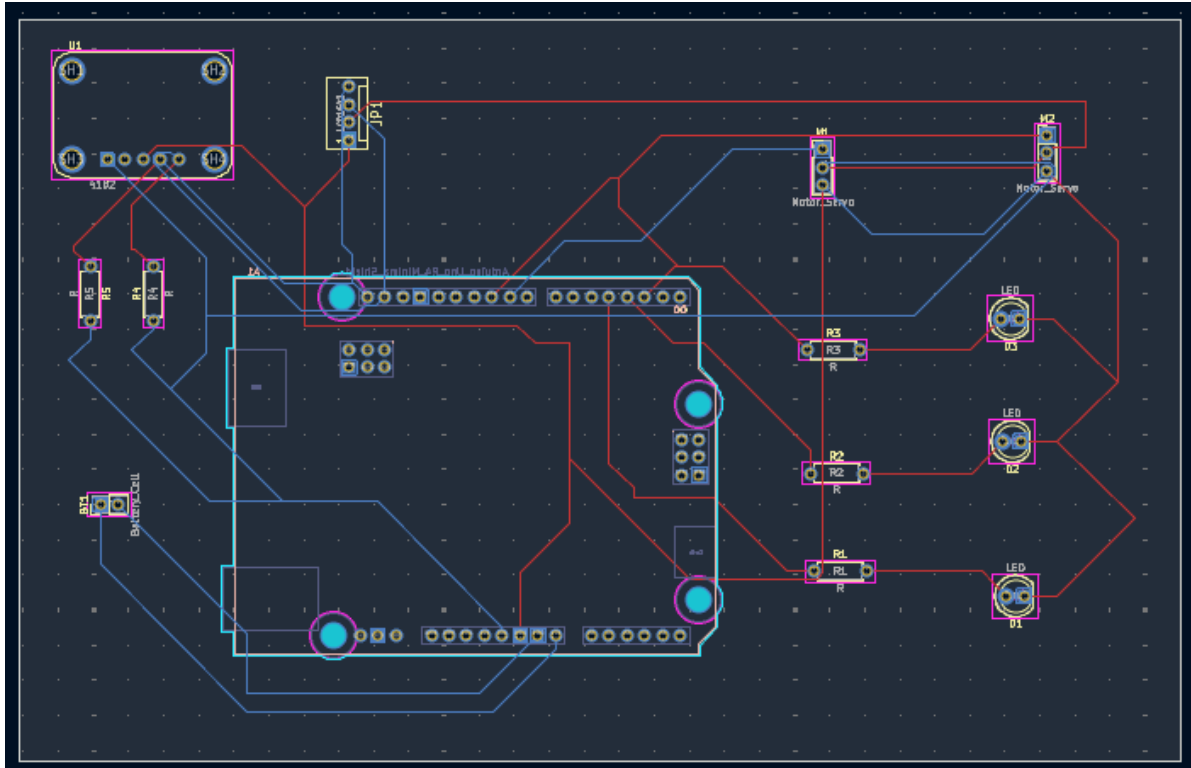


Figura 5: Diseño de enrutamiento de la PCB propuesta. Las pistas de la capa superior se muestran en rojo y las de la capa inferior en azul.

La distribución física de los componentes sobre la tarjeta puede apreciarse en la Figura 6. Esta vista superior del modelo CAD permite verificar la ubicación de los conectores, terminales y componentes electrónicos antes de la manufactura de la tarjeta, así como su compatibilidad geométrica con el Arduino UNO R4 sobre el que se montaría.

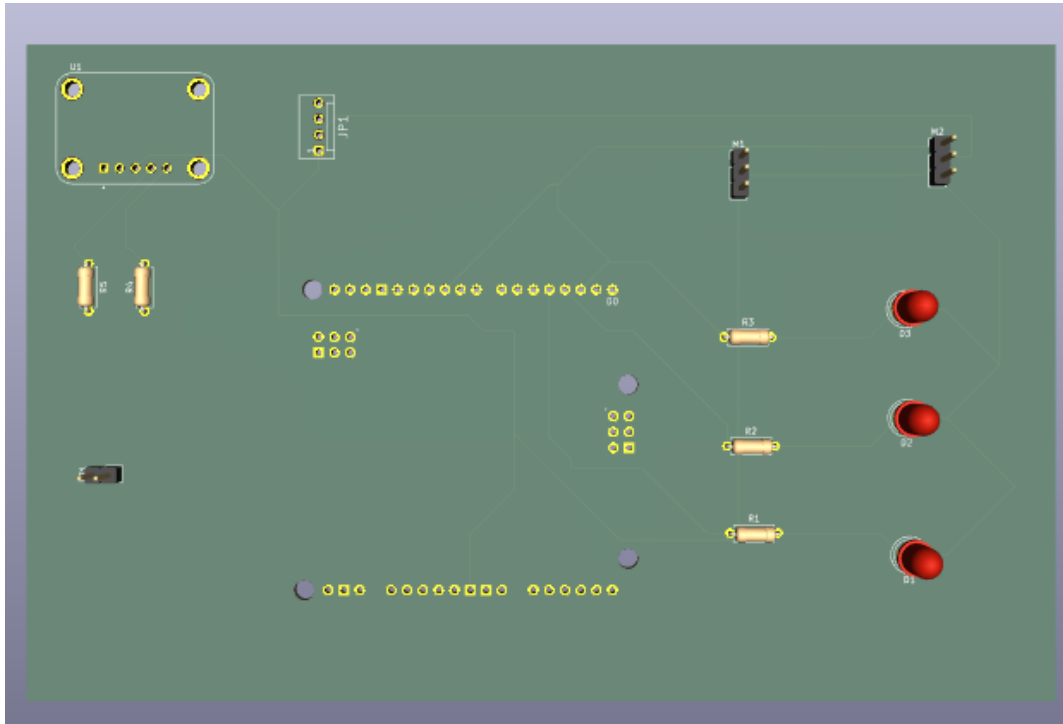


Figura 6: Vista superior del modelo CAD de la PCB propuesta tipo *shield* para Arduino UNO R4.

Finalmente, aunque la PCB no fue fabricada durante el desarrollo de la práctica, el diseño presentado en las Figuras 5 y 6 constituye una propuesta viable para futuras iteraciones del proyecto, permitiendo sustituir el cableado sobre protoboard por una solución más robusta y permanente.

5. Descripción del Hardware

El sistema se compone de un conjunto de elementos electrónicos, electromecánicos y estructurales. La Tabla 2 resume cada componente y su función dentro del sistema.

Tabla 2: Componentes del sistema y su función.

Componente	Función en el sistema
Arduino UNO R4 WiFi	Control central y transmisión de datos.
VEML7700	Medición de iluminancia en lux.
LCD 16×2 (I2C)	Visualización local de lecturas.
LEDs (rojo/amarillo/verde)	Indicador visual del rango de luminancia.
Servomotor MG995	Primer grado de libertad del brazo.
Servomotor SG90	Segundo grado de libertad del brazo.
Capacitor 1000 μ F	Estabilización de la alimentación de los servos.
Resistencias 10 k Ω	Pull-up para el bus I2C (SDA/SCL).
Batería 9 V	Alimentación del Arduino.
4×AA (6 V)	Alimentación de los servomotores.
Estructura 3D (PLA)	Soporte mecánico y carcasa.
Protoboard + Jumpers	Conexión y prototipado del circuito.

La interconexión de los distintos elementos electrónicos se muestra en la Figura 7. En ella se observan las conexiones de alimentación, comunicación y control entre el Arduino UNO R4 WiFi, el sensor de iluminancia, la pantalla LCD, los indicadores LED y los servomotores.

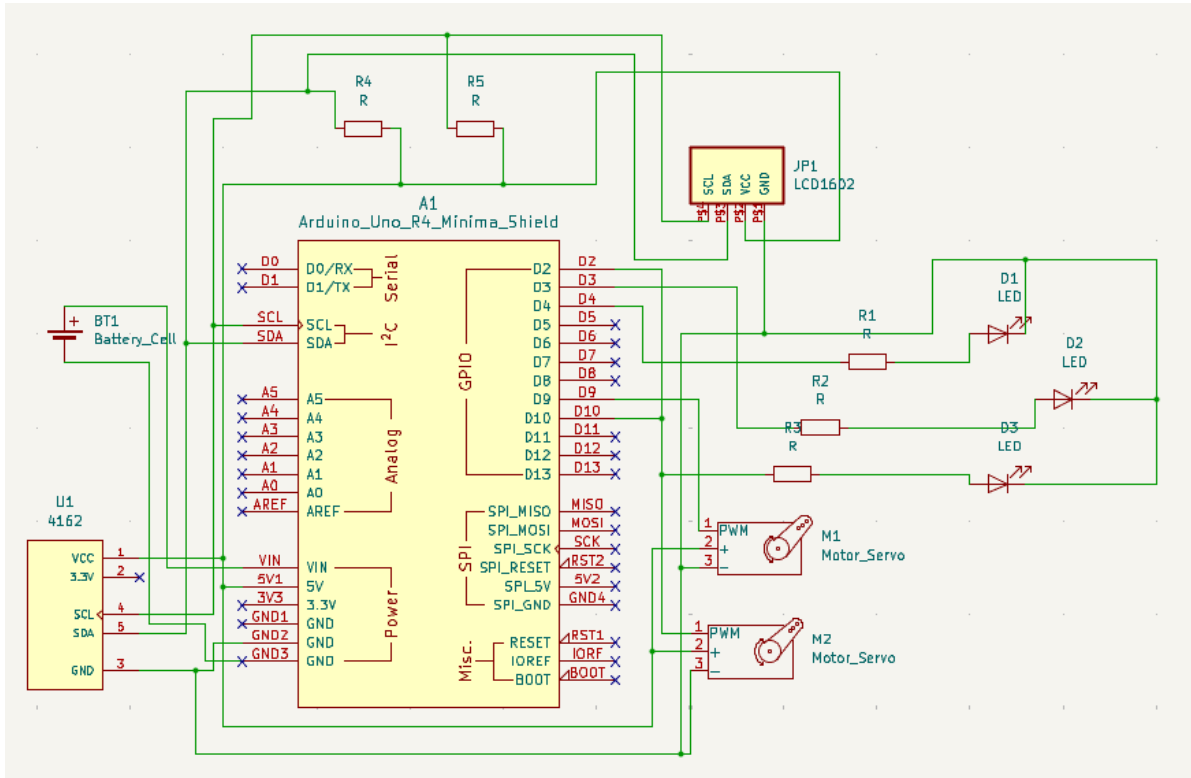


Figura 7: Esquema eléctrico completo del sistema. Se muestran las conexiones de alimentación, comunicación I2C y control entre los distintos componentes del prototipo.

La asignación de pines utilizada en el sistema se resume en la Tabla 3.

Tabla 3: Asignación de pines del Arduino UNO R4 WiFi.

Pin Arduino	Dispositivo conectado
SDA	VEML7700 y LCD 16×2 (bus I2C)
SCL	VEML7700 y LCD 16×2 (bus I2C)
D2	LED verde
D3	LED amarillo
D4	LED rojo
D9	Servomotor MG995
D10	Servomotor SG90
5 V	Alimentación del VEML7700 y LCD
GND	Tierra común del sistema

El Arduino UNO R4 WiFi actúa como controlador central: recibe los comandos de la PC,

gobierna los servomotores, lee el sensor y administra los indicadores. Como se aprecia en la Figura 7, el sensor VEML7700 y la pantalla LCD utilizan comunicación I2C y comparten el mismo bus de datos. Las terminales SDA de ambos dispositivos se conectan en paralelo al pin SDA del Arduino, mientras que las terminales SCL se conectan al pin SCL. De igual forma, ambos módulos reciben alimentación mediante las líneas de 5 V y GND del Arduino.

El sensor VEML7700 es el elemento de medición encargado de determinar la iluminancia incidente en unidades de lux. La pantalla LCD y los tres LEDs forman el sistema de indicación local, permitiendo conocer el estado de la medición sin necesidad de la PC. Los LEDs se conectan a los pines digitales D2, D3 y D4, correspondientes a los indicadores verde, amarillo y rojo, respectivamente.

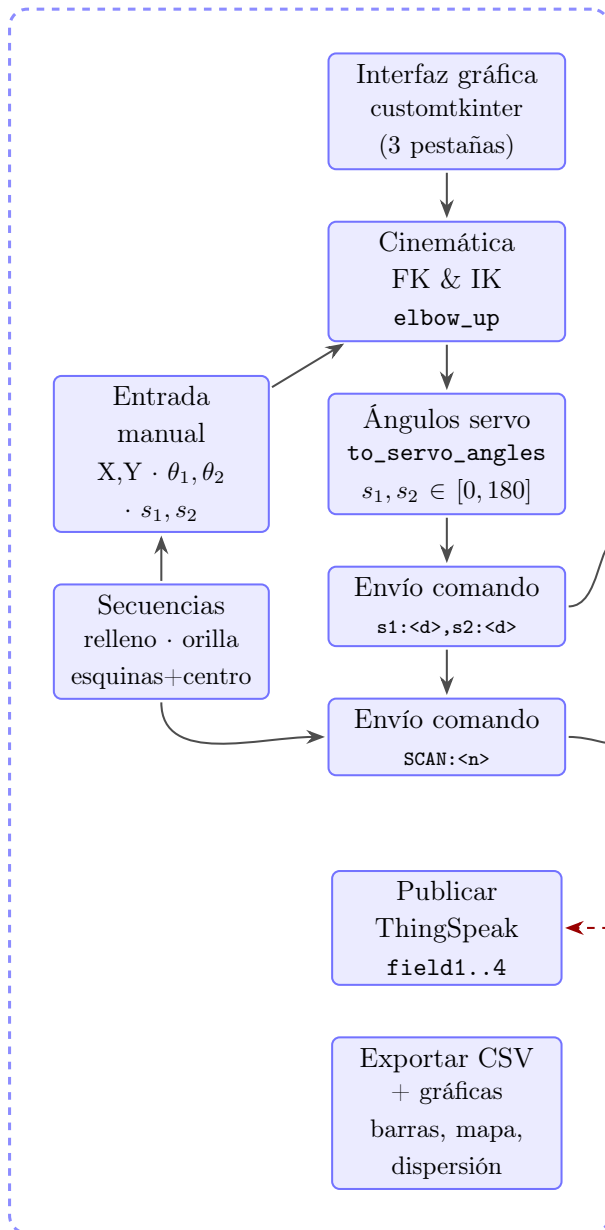
Los servomotores MG995 y SG90 proporcionan, respectivamente, el primer y segundo grado de libertad del brazo robótico. Sus señales de control se conectan a los pines D9 y D10 del Arduino, respectivamente. Debido a los requerimientos de corriente de estos actuadores, ambos se alimentan mediante un arreglo independiente de cuatro baterías AA conectadas en serie (6 V). Sin embargo, la tierra de esta fuente se conecta a la tierra del Arduino para establecer una referencia común para las señales PWM de control.

Finalmente, el capacitor de 1000 μF se conecta en paralelo con la alimentación de los servomotores para amortiguar las caídas de voltaje producidas por los picos de corriente durante el movimiento, contribuyendo a la estabilidad eléctrica del sistema.

6. Explicación del Software desarrollado

El software desarrollado está compuesto por dos programas. El primer software es una aplicación desarrollada en Python la cual al ser ejecutada permite controlar y mandar cualquier señal al brazo robótico que se ha prototipado. El segundo software, es el código de arduino, el cual directamente controla componentes como servomotores, sensor de luz, pantalla LCD y LEDs. La Figura 8 muestra el funcionamiento general del sistema y la relación entre ambas capas.

Código Python — PC



Código Arduino — Hardware

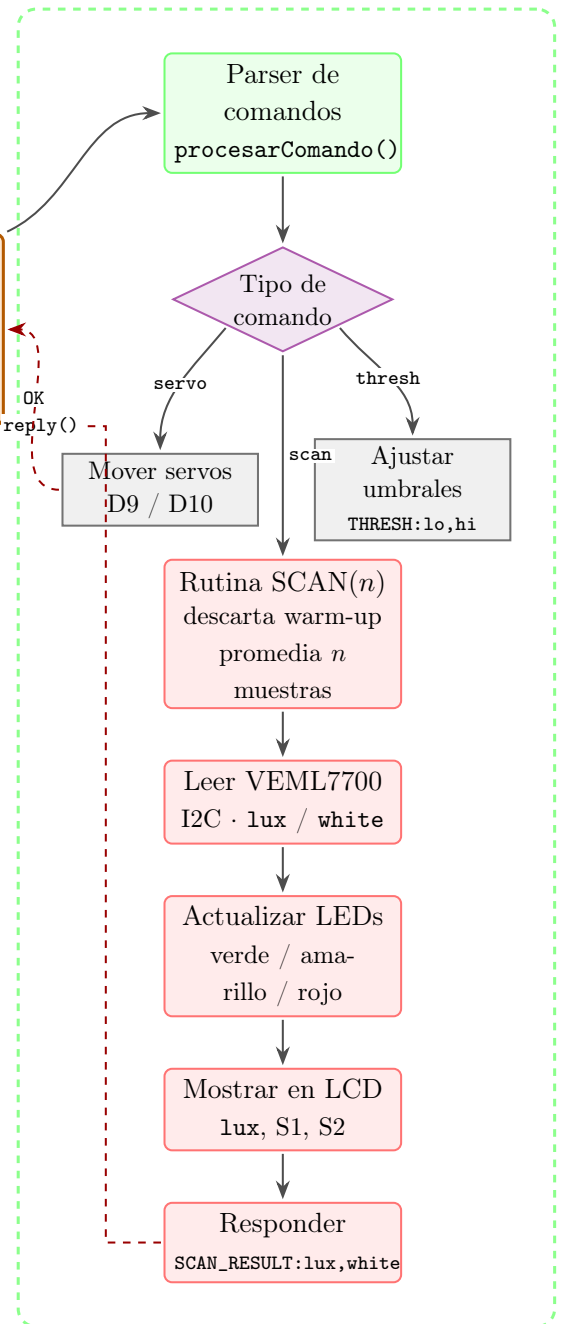


Figura 8: Diagrama general del funcionamiento del software y hardware desarrollado.

La comunicación entre la aplicación y el Arduino se ha realizado mediante un protocolo denominado texto plano. Dicho protocolo, es una forma sencilla de comunicación donde los comandos y respuestas se mandan como texto legible, no como datos binarios que pueden resultar difíciles de interpretar.

Este mismo protocolo viaja sin cambios tanto por el cable USB También llamado puerto Serial como por la red WiFi TCP, de modo que la aplicación puede usar cualquiera de los dos medios físicos de manera transparente. Los mensajes intercambiados se resumen en la Tabla 4.

Tabla 4: Protocolo de comunicación entre la PC y el Arduino.

Sentido	Mensaje	Significado
PC → Arduino	s1:<g>,s2:<g>	Mueve los servos a los ángulos indicados (0–180°).
PC → Arduino	SCAN:<n>,<ms>	Toma n muestras del sensor con un retardo entre ellas y devuelve el promedio.
PC → Arduino	THRESH:<lo>,<hi>	Ajusta los umbrales de los LEDs indicadores.
PC → Arduino	PING	Comando de diagnóstico.
Arduino → PC	SCAN_RESULT:lux,white	Promedio de iluminancia y canal blanco.
Arduino → PC	SAMPLE:<lux>	Cada muestra cruda durante un escaneo.
Arduino → PC	PONG / OK.../ ERR...	Respuestas de confirmación o error.

6.1. Aplicación de escritorio

La aplicación es la interfaz con la que el usuario opera el brazo robótico. Se calcula la cinemática inversa, se generan las trayectoras, dibuja la simulación, envía comandos al Arduino y grafica los datos del sensor.

Para el desarrollo de la aplicación se ha utilizado las siguientes librerías.

6.1.1. Librerías utilizadas

- **customtkinter / tkinter:** Tkinter es una biblioteca de Python, la cual es útil para la creación y el desarrollo de aplicaciones de escritorio [4]. Por lo que todos las pestañas, botones y ventanas han sido creadas con dicha librería.

- **numpy:** En dicho programa se ha creado el cálculo numérico y operaciones trigonométricas empleadas en la cinemática [5].
- **matplotlib:** Se ha empleado matplotlib para generar las gráficas que aparecen dentro de la aplicación [6]. Esta librería fue importante porque permitió visualizar tanto el movimiento del brazo como el área de trabajo y los datos obtenidos por el sensor.

Además de la librería principal, también se usaron algunos módulos específicos:

- **backend TkAgg:** Permite integrar las gráficas de matplotlib dentro de la ventana de Tkinter, sin que se abran en una ventana externa.
 - **Figure:** Se utilizó para crear la figura donde se dibuja cada gráfica.
 - **FigureCanvasTkAgg:** Permite mostrar la gráfica dentro de la interfaz, además permite detectar las interacciones del usuario, por ejemplo cualquier clic.
 - **Patches:** Se ha empleado para dibujar elementos visuales del sistema. Principalmente para el área alcanzable del brazo y pantalla que se desea analizar.
 - **Animation:** Ha permitido representar el movimiento del brazo de forma animada, actualizando su posición cuadro por cuadro.
- **pyserial:** Dicha librería permite la comunicación por USB y detección automática de los puertos disponibles [7].
 - **socket:** Esta ha sido de las librerías más importantes del software, pues permite la comunicación entre dos programas usando una red. En el proyecto se ha utilizado para comunicar la aplicación con el código de Arduino mediante WiFi. [8].
 - **threading:** Esta librería se ha utilizado para ejecutar tareas en segundo plano, con el propósito de no congelar la interfaz de la aplicación [9]. Es importante porque ciertas acciones, como mover el brazo robótico, esperar respuestas del Arduino o realizar varias mediciones del sensor, pueden tardar algunos segundos. Por lo que gracias a la librería, la aplicación puede seguir respondiendo mientras se ejecutan dichos procesos.
 - **urllib:** Es una librería para trabajar con direcciones web. En el presente proyecto se ha utilizado para enviar datos a ThingSpeak. Una vez que la app recibe las mediciones del sensor. [10, 11].
 - **csv, os, time, datetime:** Estas últimas librerías han sido empleadas para la exportación de resultados y manejo de tiempos de espera [12–15].

6.1.2. Cinemática del brazo

El brazo robótico se modeló como un sistema de dos grados de libertad, formado por dos brazos de longitudes L_1 y L_2 , como se ilustra en la Figura 9. El primer ángulo, θ_1 , corresponde al giro del hombro, mientras que θ_2 representa el giro relativo del segundo eslabón respecto

al primero. Antes de aplicar la cinemática, la aplicación transforma el punto seleccionado en la pantalla al sistema de referencia del brazo. De esta forma, una coordenada de la pantalla puede interpretarse como una posición física alcanzable por el extremo del mecanismo. La cinemática directa permite obtener la posición del extremo del brazo a partir de los ángulos articulares. Para ello se suman las proyecciones de ambos eslabones sobre los ejes x y y :

$$x_{\text{ext}} = x_b + L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2), \quad (7)$$

$$y_{\text{ext}} = y_b + L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2). \quad (8)$$

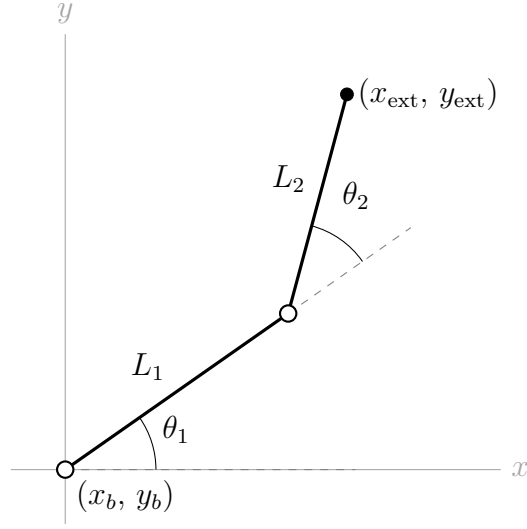


Figura 9: Modelo cinemático del brazo robótico de dos grados de libertad.

Por otro lado, la cinemática inversa resuelve el problema contrario: a partir de una posición objetivo (x, y) , se calculan los ángulos necesarios para llegar a ella. Primero se define:

$$X = x - x_b, \quad Y = y - y_b, \quad r = \sqrt{X^2 + Y^2}. \quad (9)$$

El punto solamente puede alcanzarse si cumple:

$$|L_1 - L_2| \leq r \leq L_1 + L_2. \quad (10)$$

Después, mediante ley de cosenos, se obtiene el ángulo del codo:

$$\cos \theta_2 = \frac{X^2 + Y^2 - L_1^2 - L_2^2}{2L_1L_2}. \quad (11)$$

Como existen dos soluciones posibles para una misma posición, la aplicación permite seleccionar la configuración de codo arriba o codo abajo. Finalmente, el ángulo del hombro se calcula como:

$$\theta_1 = \text{atan2}(Y, X) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2). \quad (12)$$

Una vez obtenidos θ_1 y θ_2 , estos se convierten a ángulos de servomotor dentro del rango 0° – 180° , aplicando los ajustes necesarios para compensar el montaje físico. El desarrollo matemático completo se presenta en el Anexo B.

6.1.3. Capa de transporte (USB / WiFi)

Con el objetivo que la aplicación no dependa directamente del medio de comunicación, se ha diseñado una capa de transporte. Dicha capa permite que el resto del programa envíe y reciba datos de la misma forma, sin importar si la conexión se realiza por USB o por WiFi.

Esta idea se basa en el principio de modularidad, donde cada parte del programa debe encargarse de una tarea específica [16]. En este caso, se definió una clase base llamada **BaseTransport**, que establece una interfaz común con métodos como **write**, **readline** y **close**. Esto está directamente relacionado con el concepto de clases base abstractas en Python, las cuales permiten definir una interfaz que después puede ser implementada por otras clases [17].

Dados los conceptos anteriores se implementaron dos formas de comunicación:

- **SerialTransport:** Encargado de la comunicación por USB utilizando la librería **pyserial**.
- **TcpTransport:** Encargado de la comunicación por WiFi mediante una conexión TCP.

Gracias a esta estructura, la lógica principal de la aplicación no necesita cambiar dependiendo del tipo de conexión. La aplicación siempre utiliza los mismos métodos para enviar comandos y leer respuestas, mientras que cada clase de transporte se encarga de manejar los detalles específicos de USB o WiFi.

6.1.4. Estructura de la interfaz

La interfaz tiene la siguiente estructura: En la pestaña de simulación, cuenta con un panel de configuración, donde se pueden modificar parámetros tales como las longitudes de los brazos o dar la instrucción de barrido. Además, cuenta con un área de visualización donde se aprecian los movimientos del brazo robótico. Y por último se tienen tres pestañas, simulador, análisis de ángulo y datos del sensor. En la figura 10, se muestra dicha interfaz.

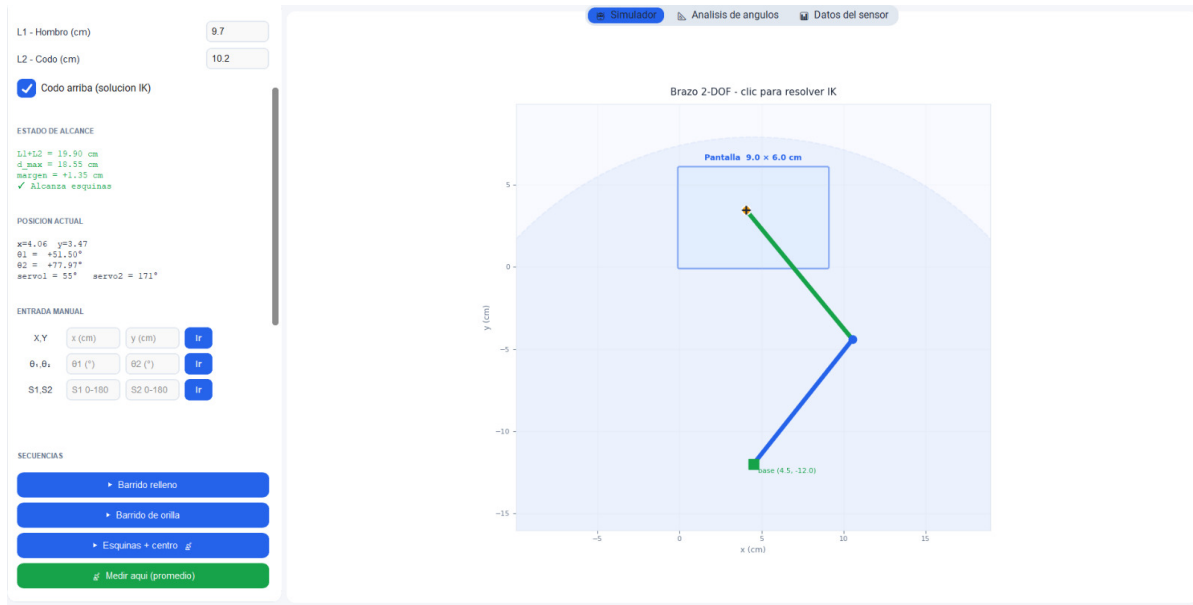


Figura 10: Interfaz de la aplicación de escritorio: panel de configuración (izquierda) y pestañas de visualización (derecha).

Panel de Configuración:

- **Área de trabajo:** Se indican las dimensiones de pantalla, objetivo, ancho, alto y separación de la base.
- **Eslabones:** Se indican las longitudes L_1 y L_2 respectivamente.
- **Estado de alcance:** Indica si el brazo puede alcanzar todas las esquinas del área de trabajo.
- **Entrada manual:** permite mover el brazo en tres modos: por coordenadas X, Y , por ángulos θ_1, θ_2 o directamente por ángulos de servo S_1, S_2 .
- **Secuencias:** Se incluyen botones para ejecutar trayectorias automáticas.
- **Conexión Arduino:** Se agrega opción al usuario de seleccionar si será por medio Serial o WiFi.
- **ThingSpeak:** Además se incluye publicación opcional de las mediciones a la nube.

Pestañas de visualización:

1. **Simulador:** Dibuja el brazo y el área de trabajo. Al hacer clic sobre un punto se resuelve la cinemática inversa y se envía la orden a los servos.
2. **Análisis de ángulos:** Grafica los ángulos θ_1 y θ_2 a lo largo de un barrido completo, marcando las zonas que superarían los límites de los servomotores.

3. **Datos del sensor:** Muestra las mediciones del VEML7700 en tres gráficas: barras de iluminancia y luminancia por punto, un mapa espacial coloreado según la luz medida y una dispersión que compara ambas magnitudes.

6.1.5. Secuencias automáticas y medición

La aplicación incluye trayectorias predefinidas que se ejecutan en un hilo independiente para no bloquear la interfaz:

- **Barrido relleno:** recorre toda el área en zig-zag.
- **Barrido de orilla:** recorre el perímetro del área.
- **Esquinas + centro:** se detiene en las cuatro esquinas y el centro y, en cada punto, realiza una medición de luz.

En los puntos de medición, la aplicación envía el comando **SCAN**, espera el promedio devuelto por el Arduino y convierte la iluminancia (en lux) a luminancia (en cd/m^2) dividiéndola por el ángulo sólido del sensor. Los resultados pueden exportarse a un archivo CSV y, opcionalmente, publicarse en la plataforma ThingSpeak.

6.2. Firmware del Arduino

El firmware se ejecuta en la placa Arduino UNO R4 WiFi y se encarga de interpretar los comandos recibidos y de actuar sobre el hardware. La versión empleada funciona únicamente por cable USB, lo que evita los picos de corriente del módulo WiFi que podían afectar la alimentación de la pantalla LCD.

6.2.1. Librerías utilizadas

- **Wire.h:** manejo del bus I2C [18].
- **Adafruit_VEML7700.h:** controlador del sensor de luz VEML7700; permite leer la iluminancia y configurar su ganancia y tiempo de integración [19].
- **LiquidCrystal_I2C.h:** control de la pantalla LCD de 16×2 caracteres a través del bus I2C [20].
- **Servo.h:** generación de la señal de control *PWM* de los servomotores [21].

6.2.2. Funcionamiento del programa

El programa de Arduino se organiza en torno a dos funciones principales:

- **setup():** inicializa la comunicación serial (9600 baud), el bus I2C, los pines de los LEDs, la pantalla LCD, el sensor (con reintentos para no bloquearse si falla) y los servomotores.

- **loop():** se ejecuta de forma continua. Primero atiende y vacía todos los comandos pendientes del puerto serial (para que el brazo responda al objetivo más reciente sin acumular retraso) y, solo cuando el brazo está quieto, actualiza la pantalla LCD y vuelve a leer el sensor.

La función `procesarComando()` identifica el tipo de mensaje recibido (movimiento de servos, escaneo, ajuste de umbrales o diagnóstico) y actúa en consecuencia. La rutina de escaneo (`ejecutarScan()`) toma el número de muestras solicitado, descarta la primera lectura (de estabilización), promedia el resto y devuelve el resultado, enviando además cada muestra individual a la PC.

El firmware también gestiona tres LEDs indicadores (verde, amarillo y rojo) que señalan si el nivel de luz medido está dentro, por debajo o por encima del rango deseado, y que se pueden reconfigurar sin necesidad de reiniciar el sistema con el comando **THRESH**. Adicionalmente, incorpora mecanismos de robustez: reintenta la conexión con el sensor si este se desconecta del bus y reinicia la pantalla LCD para recuperarse de la corrupción provocada por el ruido eléctrico en el bus I2C.

6.3. Conceptos de Arduino y comunicaciones

Para comprender el funcionamiento del sistema conviene revisar algunos conceptos clave de comunicación y hardware empleados en el proyecto.

Comunicación serial (UART) y baud rate. La PC y el Arduino intercambian datos byte a byte a través del cable USB mediante comunicación serial asíncrona. La velocidad de transmisión (baud rate) debe ser la misma en ambos extremos; en este proyecto se utiliza 9600 baudios.

Bus I2C (Inter-Integrated Circuit). El I2C es un bus de comunicación que utiliza únicamente dos líneas: SDA (datos) y SCL (reloj). Su gran ventaja es que permite conectar varios dispositivos en paralelo sobre las mismas dos líneas, distinguiéndolos por una dirección única. En este proyecto, tanto la pantalla LCD (dirección 0x27) como el sensor VEML7700 (dirección 0x10) comparten el mismo bus I2C conectado a los pines A4 (SDA) y A5 (SCL) del Arduino. Esto explica por qué con solo dos cables de datos es posible controlar ambos dispositivos simultáneamente.

Sensor de luz VEML7700. Es un sensor de luz ambiental que se comunica por I2C y entrega la iluminancia en lux, además de un canal “blanco”. Se configuran su ganancia y su tiempo de integración para adaptar la lectura a las condiciones de luz.

Servomotores y PWM. La posición angular de cada servomotor se determina mediante una señal de modulación por ancho de pulso (PWM) que genera la librería **Servo**. Cada ángulo (0–180°) corresponde a un ancho de pulso específico.

Comunicación TCP frente a serial. Mientras que la comunicación serial es un flujo continuo de bytes punto a punto, la comunicación por red (TCP) puede fragmentar o agrupar los mensajes durante su envío. Por ello, la capa de transporte WiFi de la aplicación reconstruye las líneas completas antes de procesarlas, garantizando que los comandos y respuestas se interpreten correctamente.

7. Caracterización Experimental del Sensor de Iluminancia

Una vez construida la electrónica, mecánica y programación del software. Se realizó la caracterización experimental del sensor VEML7700 con los siguientes objetivos:

- Estudiar la respuesta de una fuente de luminancia con carácter lineal, respecto a una pantalla no lineal.
- Verificar la estabilidad y repetibilidad del montaje.
- Caracterizar diferentes modelos de aproximación dada la luminancia. Es decir, se esperan modelos de regresión lineal, cuadráticos y de potencia.

7.1. Respuesta no lineal de la pantalla

Se midió la luminancia de una pantalla de computadora variando su nivel de brillo de 0 % a 100 %. Los datos experimentales muestran que la respuesta de la pantalla no es lineal: la luminancia crece de forma más pronunciada conforme aumenta el nivel de brillo. Para describir este comportamiento se compararon tres modelos de ajuste, cuyo desempeño se resume en la Tabla 5.

Tabla 5: Comparación de modelos de ajuste para la respuesta de la pantalla.

Modelo	Expresión	R^2
Lineal	$L = a x + b$	0.797
Cuadrático	$L = a x^2 + b x + c$	0.967
Potencia	$L = L_0 + k x^\gamma$	0.992

El mejor ajuste se obtuvo con el modelo de potencia

$$L = L_0 + k x^\gamma, \quad (13)$$

con un exponente $\gamma \approx 2.213$ y un coeficiente de determinación $R^2 = 0.992$, claramente superior al modelo lineal ($R^2 = 0.797$) y al cuadrático ($R^2 = 0.967$). En las Figuras 11 y 12 se observa el ajuste de los modelos ante la respuesta de la pantalla.

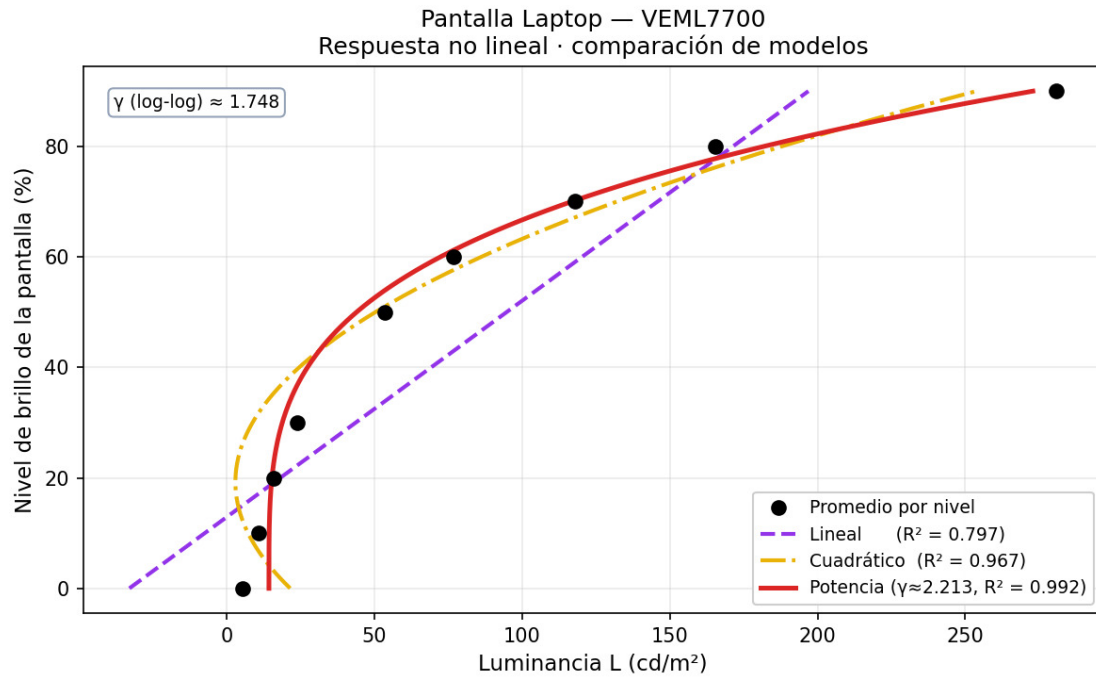


Figura 11: Respuesta no lineal - VEML7700

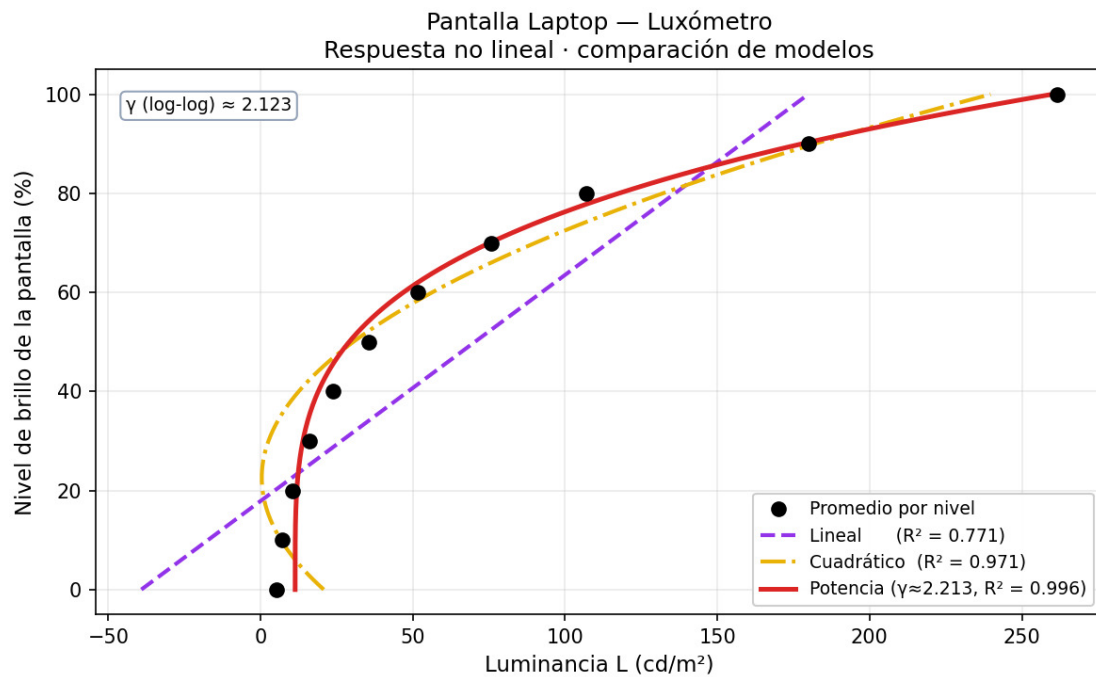


Figura 12: Respuesta no lineal - Luxómetro

7.2. Linealización logarítmica

Para estimar el exponente γ del modelo de potencia se aplicó una linealización logarítmica. Tomando logaritmo natural en ambos lados de la ecuación (13) (con L_0 ya descontado) se obtiene una relación lineal:

$$\ln(L - L_0) = \ln(k) + \gamma \ln(B), \quad (14)$$

donde B es el nivel de brillo de la pantalla. En esta forma, γ corresponde a la pendiente de la recta en el espacio log-log. El ajuste de los datos transformados confirma el valor $\gamma \approx 3.17$ y $\gamma \approx 3.23$ respectivamente. Dicho ajuste se puede apreciar en las Figuras 13 y 14.

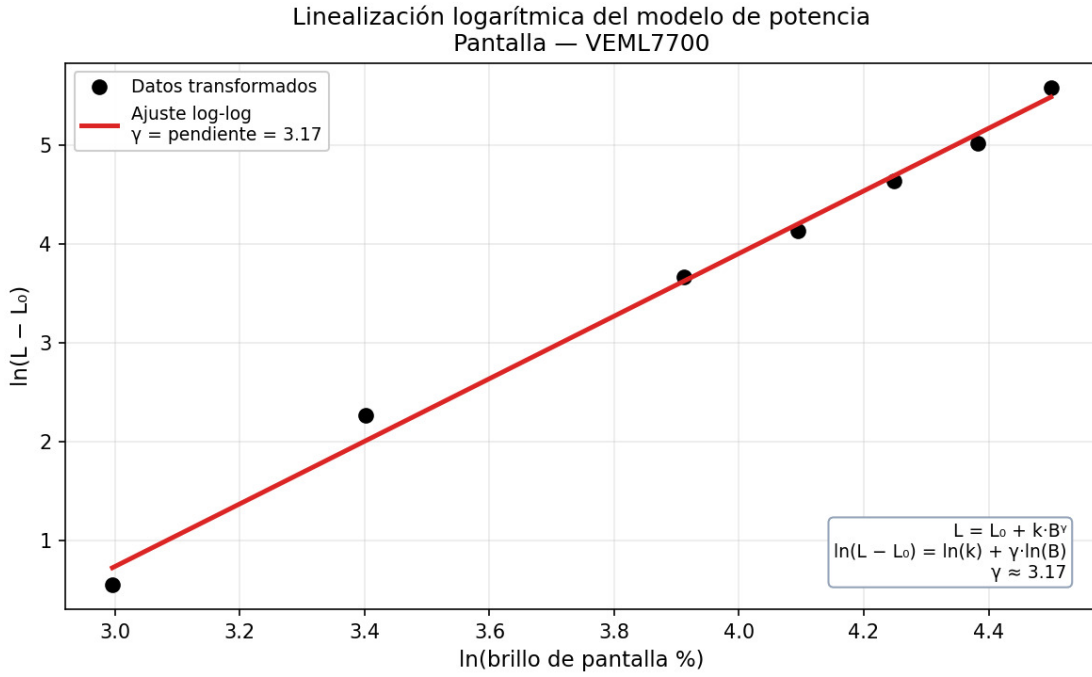


Figura 13: Linealización logarítmica del modelo de potencia. Ajuste de $\gamma \approx 3.17$

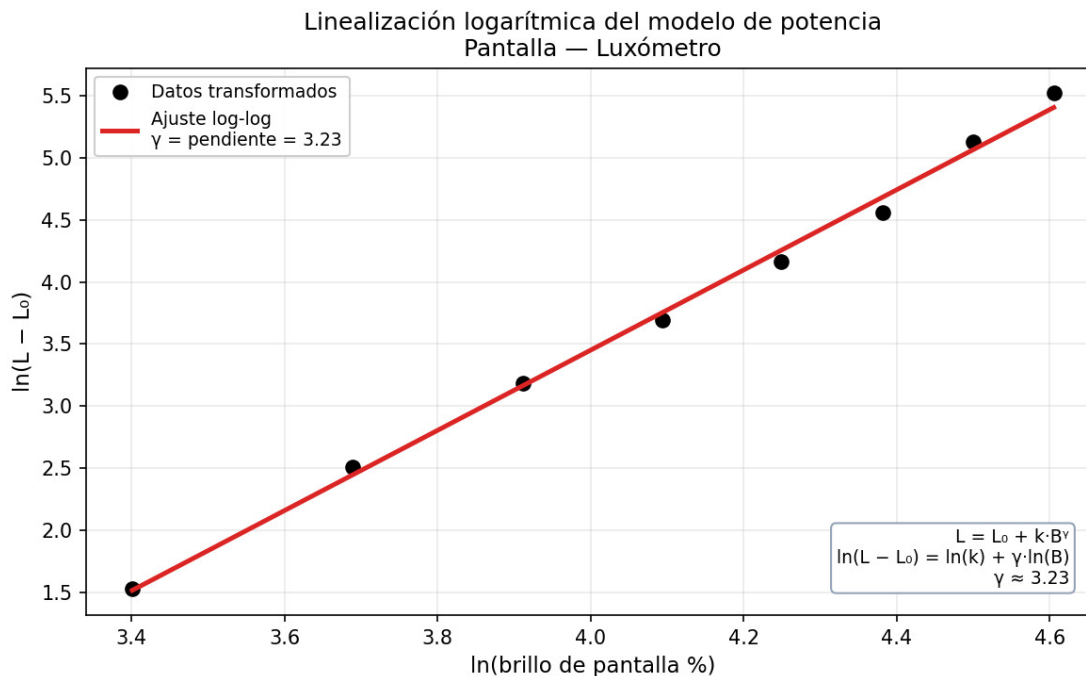


Figura 14: Caracterización con LED AZUL - Luxómetro - Luminancia

7.3. Validación del sensor con una fuente patrón de LED

Para validar el sensor se realizó un barrido de subida (0 % $\rightarrow$ 100 %) y bajada (100 % $\rightarrow$ 0 %) del brillo de una fuente LED, comparando la respuesta del sensor con un ajuste lineal. A diferencia de la pantalla, la fuente LED presenta una respuesta esencialmente lineal.

En una primera medición, el sensor mostró una linealidad del 98.31 % ($R^2 = 0.9996$). Tras aplicar un ajuste de calibración, la linealidad mejoró hasta el 99.51 % ($R^2 = 0.9999$), como se observa en las Figuras 15 y 16.

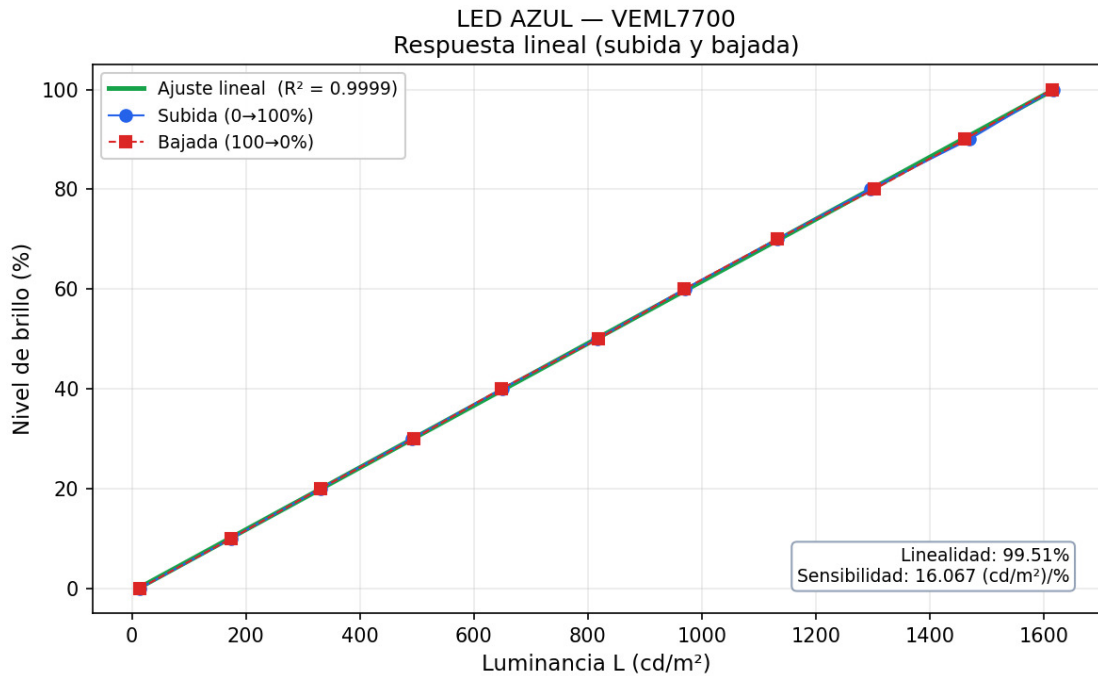


Figura 15: Caracterizacion con LED AZUL - VEML 7700 - Luminancia

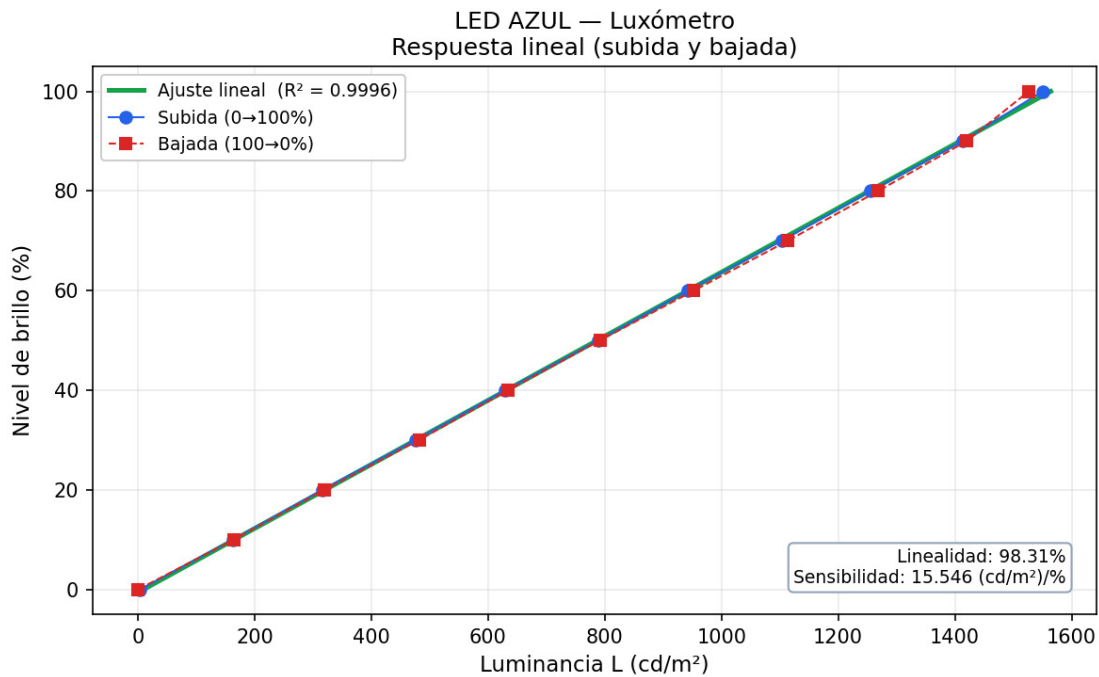


Figura 16: Caracterizacion con LED AZUL - Luxómetro - Luminancia

7.4. Parámetros metrológicos del sensor

A partir de los datos de validación se calcularon los principales parámetros metrológicos del sensor, así como el análisis de incertidumbre, resumidos en la Tabla 6

Estos resultados indican que el sensor presenta una alta linealidad (99.51 %) y una buena repetibilidad (%CV de 3.01 %), con una incertidumbre expandida U_{95} de aproximadamente 81 cd/m², dominada por la componente de Tipo B.

Tabla 6: Parámetros metrológicos obtenidos para las mediciones con LED azul y pantalla.

Parámetro	LED azul – VEML7700	LED azul – Luxómetro	Pantalla – VEML7700	Pantalla – Luxómetro
Sensibilidad ((cd/m ²)/%)	16.066704	15.546261	3.250667	2.632924
Linealidad (%)	99.509413	98.314119	69.524162	68.118833
No-linealidad (%)	0.490587	1.685881	30.475838	31.881167
Histéresis máx. (%)	0.502792	1.527067	4.860505	0.460829
Histéresis media (%)	0.150255	0.504626	2.041279	0.167574
Error absoluto medio (cd/m ²)	27.467952	6.890802	29.155773	4.017190
Error relativo medio (%)	59.287052	7.911521	47.195572	16.778342
RMSE (cd/m ²)	33.496356	9.650307	42.768633	4.890700
Repetibilidad (%CV)	2.733839	–	4.533254	–
Incert. Tipo A (cd/m ²)	4.534811	0.000000	0.529524	0.000000
Incert. Tipo B (cd/m ²)	1.013622	0.341627	1.577977	0.341627
Incert. combinada u_c (cd/m ²)	4.646713	0.341627	1.664454	0.341627
Incert. expandida U_{95} (cd/m ²)	9.293426	0.683255	3.328908	0.683255
Resolución efectiva (cd/m ²)	150.386954	121.893491	5.192959	1.775148
R^2	0.999938	0.999615	0.991680	0.996155

8. Conclusiones

El sistema desarrollado cumplió con el objetivo del reto: medir la luminancia de pantallas digitales y determinar si se encuentra dentro de un rango aceptable. El brazo robótico de 2-GDL demostró ser una solución práctica y adaptable para posicionar el sensor de forma reproducible, y el conjunto resultó portátil gracias a su alimentación por baterías y su estructura impresa en 3D.

La caracterización experimental confirmó que la respuesta de una pantalla frente a su nivel de brillo es no lineal y se describe adecuadamente mediante un modelo de potencia con exponente $\gamma \approx 2.213$ ($R^2 = 0.991$). La validación del sensor VEML7700 frente a una fuente patrón de LED arrojó una linealidad del 99.51 % tras la calibración, con una repetibilidad e incertidumbre adecuadas para la aplicación.

El mayor reto del proyecto fue la integración de hardware y software: coordinar la cinemática calculada en la PC, la comunicación con el Arduino y el control simultáneo de los servomotores, el sensor, la LCD y los LEDs sobre un bus I2C compartido y una alimentación común. La solución de estos problemas (separación de alimentaciones, resistencias pull-up, reintentos del sensor y reinicio de la LCD) fue clave para lograr un sistema estable y confiable.

8.1. Conclusiones individuales

Alfredo Almaguer

Mi responsabilidad dentro del proyecto fue el diseño del circuito eléctrico del brazo robótico, y resultó ser una de las partes que más me retó. Más allá de conectar los componentes, lo interesante fue pensar en cómo robustecer físicamente el circuito para que el sistema funcionara de forma confiable durante las mediciones. Incorporar baterías como alimentación externa de los servomotores me permitió aislar su consumo del resto de la electrónica y evitar que las demandas de corriente afectaran al control. De igual manera, agregar un capacitor como medida de protección frente a los picos de corriente y las caídas de voltaje resultó clave para estabilizar el comportamiento del brazo. Este proceso me dejó una apreciación mucho más clara de que un buen diseño electrónico no solo debe funcionar en condiciones ideales, sino anticiparse a los problemas reales de operación.

Feliciano Gutiérrez

Definitivamente la parte del proyecto que más disfruté fue la transformación de coordenadas que conecta al espacio de la pantalla con el sistema de referencia físico del brazo. Me pareció muy interesante ver cómo un punto seleccionado en la interfaz podía traducirse en una posición real alcanzable por el mecanismo, y cómo las matemáticas detrás de ese paso le daban sentido a todo el sistema. También me resultó muy gratificante conectar el brazo con una aplicación sencilla de utilizar, porque me obligó a pensar no solo en que las cosas funcionaran, sino en que cualquier persona pudiera operarlas con facilidad. Este trabajo me ayudó a entender el valor de unir el rigor técnico con una experiencia de usuario clara y accesible.

Héctor Campbell

Mi parte favorita del proyecto fue el diseño de las piezas en 3D que conforman la estructura del brazo. Disfruté mucho el proceso de considerar las medidas importantes de cada componente y traducirlas en piezas que encajaran correctamente para dar forma a la arquitectura física del mecanismo. Fue especialmente satisfactorio ver cómo un conjunto de piezas diseñadas con cuidado terminaba integrándose en una estructura funcional y estable. Este trabajo me enseñó la importancia de la precisión dimensional y de pensar en el ensamble desde las primeras etapas del diseño, ya que pequeños detalles podían marcar la diferencia en el resultado final. Como mejora a futuro, propongo el diseño de una PCB que integre las conexiones del sistema, de manera que se eviten los falsos contactos y se asegure de forma más confiable la unión entre los componentes, dándole mayor robustez y limpieza al conjunto electrónico del brazo.

Bernardo Lugo

La etapa que más disfruté fue la caracterización del sensor, donde mi labor consistió en tomar mediciones con el VEML7700 y compararlas con las obtenidas mediante un luxómetro de referencia. Este proceso me permitió evaluar de manera directa la fiabilidad del sensor y comprender hasta qué punto sus lecturas reflejaban valores reales de brillo. Me resultó muy enriquecedor el aspecto experimental de validar los datos, pues me hizo apreciar que ningún instrumento debe darse por confiable sin antes contrastarlo con una referencia adecuada. Gracias a esta experiencia reforcé mi criterio para interpretar mediciones y para distinguir entre un resultado aparentemente correcto y uno verdaderamente validado.

Referencias

- [1] Amazon México. (2022, Jan.) Pantalla lcd para carrier transicold apx display module 12-00663-64. Amazon México. ASIN: B09QBJXTTN; referencia del fabricante: 12-00663-64. [Online]. Available: <https://www.amazon.com.mx/Pantalla-Carrier-Transicold-Display-12-00663-64/dp/B09QBJXTTN>
- [2] International Commission on Illumination (CIE), *ILV: International Lighting Vocabulary*, CIE Std. CIE S 017/E:2020, 2020.
- [3] W. R. McCluney, *Introduction to Radiometry and Photometry*, 2nd ed. Norwood, MA, USA: Artech House, 2014.
- [4] Python Software Foundation. (2026) tkinter — python interface to tcl/tk. Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/tkinter.html>
- [5] NumPy Developers. NumPy: The fundamental package for scientific computing with python. NumPy. [Online]. Available: <https://numpy.org/>
- [6] Matplotlib Development Team. Matplotlib: Visualization with python. Matplotlib. [Online]. Available: <https://matplotlib.org/>
- [7] C. Liechti. (2020, Nov.) pyserial: Python serial port extension. Python Package Index. [Online]. Available: <https://pypi.org/project/pyserial/>
- [8] Python Software Foundation. (2026) socket — low-level networking interface. Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/socket.html>
- [9] ——. (2026) threading — thread-based parallelism. Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/threading.html>
- [10] ——. (2026) urllib.parse — parse urls into components. Python Software Foundation. [Online]. Available: <https://docs.python.org/es/3/library/urllib.parse.html>
- [11] ——. (2026) urllib.request — biblioteca extensible para abrir urls. Python Software Foundation. [Online]. Available: <https://docs.python.org/es/3/library/urllib.request.html>
- [12] ——. (2026) csv — csv file reading and writing. Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/csv.html>
- [13] ——. (2026) time — time access and conversions. Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/time.html>

- [14] ——. (2026) datetime — basic date and time types. Python Software Foundation. [Online]. Available: <https://docs.python.org/3/library/datetime.html>
- [15] ——. (2026) os — interfaces misceláneas del sistema operativo. Python Software Foundation. [Online]. Available: <https://docs.python.org/es/3.9/library/os.html>
- [16] D. L. Parnas, “On the criteria to be used in decomposing systems into modules,” *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.
- [17] Python Software Foundation, “abc — abstract base classes,” <https://docs.python.org/3/library/abc.html>, 2026, accessed: 2026-06-14.
- [18] Arduino, “Wire,” <https://docs.arduino.cc/language-reference/en/functions/communication/wire/>, s.f., consultado el 14 de junio de 2026.
- [19] Adafruit, “Adafruit_VEML7700.h File Reference,” https://adafruit.github.io/Adafruit_VEML7700/html/_adafruit___v_e_m_l7700_8h.html, s.f., documentación de la librería Adafruit VEML7700 Lux Sensor Library. Consultado el 14 de junio de 2026.
- [20] Arduino, “LiquidCrystal I2C,” <https://docs.arduino.cc/libraries/liquidcrystal-i2c/>, s.f., documentación de la librería LiquidCrystal I2C. Consultado el 14 de junio de 2026.
- [21] —, “Servo,” <https://docs.arduino.cc/libraries/servo/>, s.f., documentación de la librería Servo. Consultado el 14 de junio de 2026.

A. Códigos desarrollados

Los códigos desarrollados para este proyecto, incluyendo la aplicación en Python y los programas utilizados en Arduino, se encuentran disponibles en la siguiente carpeta de Google Drive:

Carpeta de códigos desarrollados

B. Desarrollo matemático de la cinemática del brazo de dos grados de libertad

Se considera un brazo planar de dos eslabones L_1 y L_2 , con ángulos articulares θ_1 y θ_2 .

Transformación de coordenadas

Sea (u, v) un punto seleccionado dentro de la pantalla. Si la pantalla tiene ancho W , alto H y separación vertical a respecto a la base del brazo, se usa:

$$x = u - \frac{W}{2}, \quad (15)$$

$$y = a + v. \quad (16)$$

De forma general:

$$x = x_0 + u, \quad (17)$$

$$y = y_0 + v. \quad (18)$$

Cinemática directa

Posición del codo:

$$x_{\text{codo}} = x_b + L_1 \cos \theta_1, \quad (19)$$

$$y_{\text{codo}} = y_b + L_1 \sin \theta_1. \quad (20)$$

Posición del extremo:

$$x_{\text{ext}} = x_{\text{codo}} + L_2 \cos(\theta_1 + \theta_2), \quad (21)$$

$$y_{\text{ext}} = y_{\text{codo}} + L_2 \sin(\theta_1 + \theta_2). \quad (22)$$

Por lo tanto:

$$x_{\text{ext}} = x_b + L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2), \quad (23)$$

$$y_{\text{ext}} = y_b + L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2). \quad (24)$$

Cinemática inversa

Primero se define el punto objetivo respecto a la base del brazo:

$$X = x - x_b, \quad (25)$$

$$Y = y - y_b. \quad (26)$$

La distancia al punto objetivo es:

$$r = \sqrt{X^2 + Y^2}. \quad (27)$$

Condición de alcance:

$$|L_1 - L_2| \leq r \leq L_1 + L_2. \quad (28)$$

Por ley de cosenos:

$$r^2 = L_1^2 + L_2^2 + 2L_1L_2 \cos \theta_2. \quad (29)$$

Despejando:

$$\cos \theta_2 = \frac{X^2 + Y^2 - L_1^2 - L_2^2}{2L_1L_2}. \quad (30)$$

Para seleccionar codo arriba o codo abajo:

$$\sin \theta_2 = \sigma \sqrt{1 - \cos^2 \theta_2}, \quad (31)$$

$$\sigma = \begin{cases} +1, & \text{codo arriba,} \\ -1, & \text{codo abajo.} \end{cases} \quad (32)$$

Entonces:

$$\theta_2 = \text{atan2}(\sin \theta_2, \cos \theta_2). \quad (33)$$

Para el ángulo del hombro:

$$\gamma = \text{atan2}(Y, X), \quad (34)$$

$$\beta = \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2). \quad (35)$$

Finalmente:

$$\theta_1 = \gamma - \beta. \quad (36)$$

Es decir:

$$\theta_1 = \text{atan2}(Y, X) - \text{atan2}(L_2 \sin \theta_2, L_1 + L_2 \cos \theta_2). \quad (37)$$

Conversión a ángulos de servo

Los ángulos articulares se convierten a grados:

$$\theta_1^\circ = \theta_1 \frac{180^\circ}{\pi}, \quad (38)$$

$$\theta_2^\circ = \theta_2 \frac{180^\circ}{\pi}. \quad (39)$$

Después se ajustan al montaje físico:

$$S_1 = 90^\circ + \theta_1^\circ + \Delta_1, \quad (40)$$

$$S_2 = \theta_2^\circ + \Delta_2. \quad (41)$$

Finalmente, se limitan al rango del servomotor:

$$S_i = \min(180^\circ, \max(0^\circ, S_i)). \quad (42)$$